บทที่ 2
ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

การพัฒนาแอปพลิเคชัน Chatshop ประกอบด้วยแอปพลิเคชันบนสมาร์ตโฟนสำหรับเจ้าหน้าที่ร้านค้า และระบบบริการหลังบ้านสำหรับรับส่งข้อความกับแพลตฟอร์มภายนอก จัดเก็บข้อมูล และประมวลผลแชทบอท ผู้จัดทำจึงศึกษาแนวคิดและเทคโนโลยีที่เกี่ยวข้องทั้งสองส่วน เพื่อนำมาออกแบบและพัฒนาระบบให้ทำงานได้ตามวัตถุประสงค์ โดยมีหัวข้อดังนี้

2.1 Flutter

2.2 ภาษา Dart

2.3 Riverpod

2.4 go_router

2.5 Laravel

2.6 REST API

2.7 OAuth 2.0

2.8 LINE Messaging API

2.9 Meta Graph API

2.10 Webhook

2.11 Queue และ Task Scheduling

2.12 WebSocket และ Laravel Reverb

2.13 Firebase Cloud Messaging (FCM)

2.14 MySQL

2.15 MongoDB

2.16 สถาปัตยกรรม Multi-Tenant

2.17 การควบคุมสิทธิ์ตามบทบาท (RBAC)

2.18 แชทบอท

2.19 แบบจำลองภาษาขนาดใหญ่ (Large Language Model: LLM)

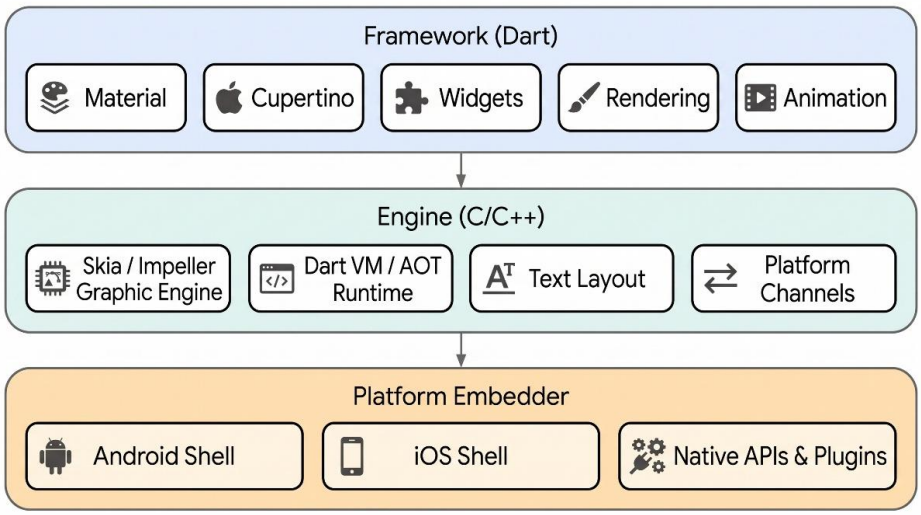
2.20 RAG (Retrieval-Augmented Generation)

2.21 Embedding และความคล้ายโคไซน์

2.22 งานวิจัยที่เกี่ยวข้อง

2.1 Flutter

Flutter คือชุดพัฒนาซอฟต์แวร์แบบโอเพนซอร์สของ Google สำหรับสร้างแอปพลิเคชันหลายแพลตฟอร์มจากโค้ดชุดเดียว [1] ส่วนติดต่อผู้ใช้ประกอบขึ้นจากวิดเจ็ตที่ซ้อนกันเป็นลำดับชั้นและวาดหน้าจอด้วยเอนจินกราฟิกของ Flutter เอง จึงแสดงผลเหมือนกันทุกอุปกรณ์ ดังรูปที่ 2.1 ระหว่างพัฒนารองรับ Hot Reload ที่แสดงผลการแก้ไขโค้ดได้ทันทีโดยไม่ต้องเริ่มแอปพลิเคชันใหม่



รูปที่ 2.1 โครงสร้างการทำงานของ Flutter

โครงสร้างการทำงานของ Flutter ตามสถาปัตยกรรมแบ่งออกเป็น 3 ชั้นหลัก ดังนี้

1) ชั้นเฟรมเวิร์ก พัฒนาด้วยภาษา Dart ประกอบด้วยชุดวิดเจ็ตมาตรฐานตามแนวทาง Material สำหรับระบบปฏิบัติการ Android และ Cupertino สำหรับระบบปฏิบัติการ iOS รวมถึงระบบการวาดภาพและภาพเคลื่อนไหวสำหรับสร้างส่วนต่อประสานกับผู้ใช้งาน

2) ชั้นเอนจิน พัฒนาด้วยภาษา C และ C++ ทำหน้าที่ประมวลผลการทำงานระดับแกนกลาง ได้แก่ การเรนเดอร์ภาพกราฟิกด้วย Skia หรือ Impeller การประมวลผลโค้ดภาษา Dart ผ่านระบบการทำงานของรหัสเครื่อง AOT ตลอดจนการจัดวางตำแหน่งข้อความและช่องทางติดต่อกับแพลตฟอร์ม

3) ชั้นเอ็มเบดเดอร์ ทำหน้าที่ประสานงานระหว่างเอนจินของ Flutter กับระบบปฏิบัติการของอุปกรณ์ โดยสร้างเชลล์สำหรับ Android หรือ iOS เพื่อให้แอปพลิเคชันเข้าถึงความสามารถของฮาร์ดแวร์และปลั๊กอินเฉพาะของอุปกรณ์ได้อย่างสมบูรณ์

2.1.1 ข้อดี

2.1.1.1 เขียนโค้ดชุดเดียวแล้วนำไปใช้ได้ทั้ง Android และ iOS

2.1.1.2 มีวิดเจ็ตตามแนวทาง Material Design ให้ใช้งานครบถ้วน

2.1.1.3 Hot Reload ช่วยลดเวลาในการพัฒนาและปรับแต่งหน้าจอ

2.1.2 ข้อเสีย

2.1.2.1 ไฟล์ติดตั้งมีขนาดใหญ่กว่าแอปพลิเคชันแบบเนทีฟ

2.1.2.2 การใช้ความสามารถเฉพาะของอุปกรณ์ต้องพึ่งแพ็กเกจเสริม

2.1.3 การนำไปใช้ในโครงการ ผู้จัดทำใช้ Flutter 3 พัฒนาแอปพลิเคชันสำหรับสมาร์ตโฟนบนระบบปฏิบัติการ Android แบ่งโค้ดเป็น 13 โมดูลตามหน้าที่ เช่น การเข้าสู่ระบบ กล่องข้อความ หน้าต่างสนทนา ลูกค้า สินค้า คำสั่งซื้อ การแจ้งเตือน รายงาน และการตั้งค่า

2.2 ภาษา Dart

Dart คือภาษาโปรแกรมเชิงวัตถุของ Google ที่ใช้พัฒนาแอปพลิเคชันด้วย Flutter [2] มีระบบชนิดข้อมูลแบบ Null Safety ที่ตรวจสอบค่าว่างตั้งแต่ขั้นตอนคอมไพล์ และรองรับการเขียนโปรแกรมแบบไม่ประสานเวลาด้วย Future Stream และคำสั่ง async กับ await ทำให้รอข้อมูลจากเครือข่ายได้โดยหน้าจอไม่ค้าง

2.2.1 ข้อดี

2.2.1.1 Null Safety ช่วยลดข้อผิดพลาดจากค่าว่างขณะแอปพลิเคชันทำงาน

2.2.1.2 คอมไพล์เป็นรหัสเครื่องสำหรับการติดตั้งจริง จึงทำงานได้รวดเร็ว

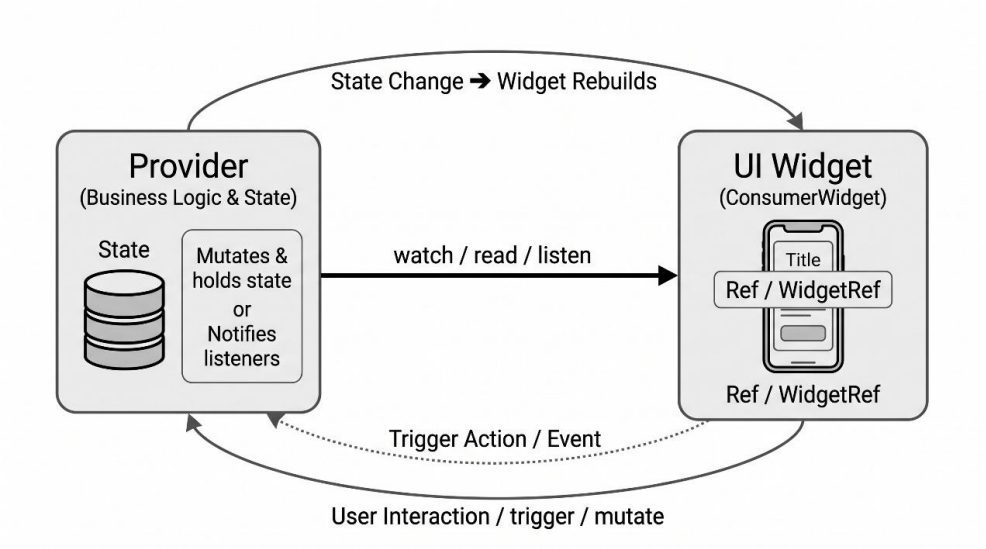
2.2.2 ข้อเสีย

2.2.2.1 มีผู้ใช้งานน้อยกว่าภาษาหลักอื่น เช่น JavaScript และ Kotlin

2.2.3 การนำไปใช้ในโครงการ ผู้จัดทำใช้ Dart เขียนโค้ดทั้งหมดของแอปพลิเคชัน โดยใช้ Future สำหรับเรียก REST API ของระบบหลังบ้าน และใช้ Stream สำหรับรับเหตุการณ์แบบเรียลไทม์และการแจ้งเตือน

2.3 Riverpod

Riverpod คือไลบรารีบริหารจัดการสถานะของแอปพลิเคชันสำหรับ Flutter [3] ข้อมูลแต่ละชุดประกาศเป็น Provider ที่ไม่ขึ้นกับตำแหน่งของวิดเจ็ต วิดเจ็ตที่ติดตามข้อมูลด้วยคำสั่ง watch จะสร้างหน้าจอใหม่เมื่อข้อมูลเปลี่ยน และ Provider หนึ่งสามารถติดตาม Provider อื่นได้ เมื่อข้อมูลต้นทางเปลี่ยน ข้อมูลที่คำนวณต่อจากกันจะถูกคำนวณใหม่ตามลำดับโดยอัตโนมัติ ดังรูปที่ 2.2



รูปที่ 2.2 ความสัมพันธ์ระหว่าง Provider และวิดเจ็ตใน Riverpod

การทำงานระหว่าง Provider และวิดเจ็ตใน Riverpod ประกอบด้วย 3 องค์ประกอบหลัก ดังนี้

1) ตัวจัดเตรียมข้อมูลและสถานะ ทำหน้าที่จัดเก็บสถานะทางตรรกะของระบบ ประมวลผลการเปลี่ยนแปลงข้อมูล และแจ้งเตือนไปยังวิดเจ็ตที่ติดตามอยู่เมื่อข้อมูลมีการเปลี่ยนแปลง

2) ส่วนต่อประสานผู้ใช้ วิดเจ็ตแบบ ConsumerWidget ใช้ตัวอ้างอิง WidgetRef ในการเฝ้าดู อ่านค่า หรือรอรับเหตุการณ์จากตัวจัดเตรียมข้อมูล เมื่อสถานะเปลี่ยนแปลง วิดเจ็ตจะถูกวาดใหม่โดยอัตโนมัติ

3) ปฏิสัมพันธ์ของผู้ใช้ เมื่อผู้ใช้งานสั่งงานผ่านหน้าจอ วิดเจ็ตจะทริกเกอร์เหตุการณ์เพื่อเรียกเมธอดในตัวจัดเตรียมข้อมูลให้ดำเนินการปรับปรุงสถานะต่อไป

2.3.1 ข้อดี

2.3.1.1 แยกตรรกะของข้อมูลออกจากหน้าจอ ทำให้ทดสอบได้ง่าย

2.3.1.2 ตรวจสอบชนิดข้อมูลได้ตั้งแต่ขั้นตอนคอมไพล์

2.3.1.3 จัดการสถานะกำลังโหลดและข้อผิดพลาดให้โดยอัตโนมัติ

2.3.2 ข้อเสีย

2.3.2.1 ผู้พัฒนาต้องเรียนรู้แนวคิดของ Provider หลายชนิดก่อนใช้งาน

2.3.3 การนำไปใช้ในโครงการ ผู้จัดทำใช้ Riverpod เก็บสถานะการเข้าสู่ระบบ ตัวเรียก REST API ข้อมูลบทสนทนา บริการเรียลไทม์ และบริการแจ้งเตือน เมื่อผู้ใช้เข้าสู่ระบบหรือออกจาก

ระบบ Provider ที่ติดตามสถานะนี้จะเชื่อมต่อหรือยกเลิกการเชื่อมต่อเรียลไทม์และการลงทะเบียนอุปกรณ์ให้เองทันที

2.4 go_router

go_router คือแพ็กเกจกำหนดเส้นทางหน้าจอของ Flutter [4] ที่ระบุแต่ละหน้าจอด้วยเส้นทางในรูปแบบ URL เช่น /inbox/thread/:id รองรับการเปลี่ยนเส้นทางตามเงื่อนไข และการซ้อนเส้นทางไว้ภายใต้โครงหน้าจอเดียวกันด้วย ShellRoute

2.4.1 ข้อดี

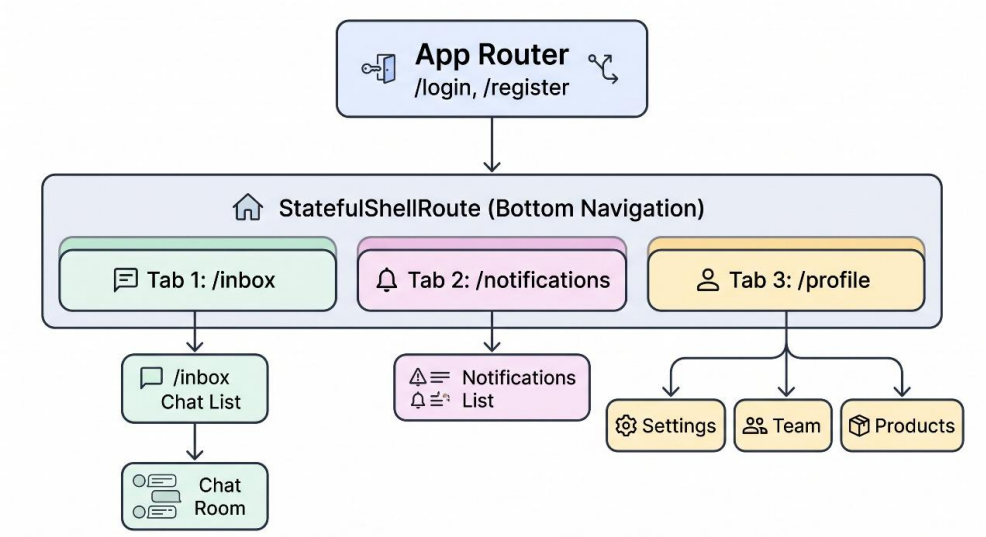
2.4.1.1 เปิดหน้าจอที่ต้องการได้โดยตรงจากเส้นทาง เช่น เมื่อแตะการแจ้งเตือน

2.4.1.2 ตรวจสอบสิทธิ์ก่อนเข้าหน้าจอได้จากจุดเดียว

2.4.2 ข้อเสีย

2.4.2.1 เส้นทางที่ซ้อนกันหลายชั้นทำให้โครงสร้างโค้ดซับซ้อนขึ้น

2.4.3 การนำไปใช้ในโครงการ ผู้จัดทำใช้ StatefulShellRoute สร้างแถบนำทางด้านล่าง 3 แท็บ ได้แก่ กล่องข้อความ การแจ้งเตือน และข้อมูลผู้ใช้ โดยแต่ละแท็บจำหน้าจอล่าสุดไว้เมื่อสลับแท็บ และกำหนดให้ผู้ใช้ที่ยังไม่เข้าสู่ระบบถูกส่งไปยังหน้าเข้าสู่ระบบเสมอ ดังรูปที่ 2.3



รูปที่ 2.3 โครงสร้างเส้นทางหน้าจอของแอปพลิเคชันด้วย go_router

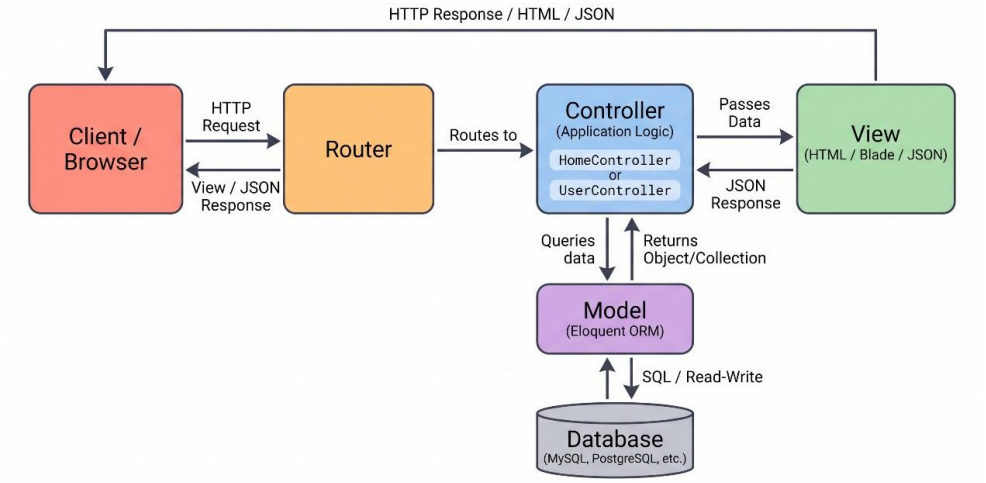
โครงสร้างการนำทางของแอปพลิเคชันแบ่งตามเส้นทางหน้าจอออกเป็น 2 ระดับหลัก ดังนี้

1) เส้นทางระดับบน จัดการการเปลี่ยนเส้นทางพื้นฐานสำหรับผู้ใช้ที่ยังไม่ได้รับการยืนยันตัวตน ได้แก่ หน้าเข้าสู่ระบบและหน้าลงทะเบียน

2) เส้นทางโครงสร้างหลัก จัดการการนำทางผ่านแถบเมนูด้านล่าง 3 แท็บ ได้แก่ แท็บกล่องข้อความที่นำทางสู่รายการแชทและห้องสนทนา แท็บการแจ้งเตือนที่แสดงรายการแจ้งเตือน และแท็บข้อมูลส่วนตัวที่สามารถเปิดหน้าจอการตั้งค่า ข้อมูลทีม และรายการสินค้า

2.5 Laravel

Laravel คือเฟรมเวิร์กภาษา PHP ที่ใช้รูปแบบ MVC (Model-View-Controller) [5] มีเครื่องมือพื้นฐานสำหรับพัฒนาระบบฝั่งเครื่องแม่ข่าย ได้แก่ การกำหนดเส้นทาง Eloquent ORM สำหรับติดต่อฐานข้อมูล ระบบยืนยันตัวตน และระบบคิวงานสำหรับประมวลผลเบื้องหลัง การทำงานตามรูปแบบ MVC แสดงดังรูปที่ 2.4



รูปที่ 2.4 การทำงานตามรูปแบบ MVC ของ Laravel

การประมวลผลคำขอตามรูปแบบสถาปัตยกรรม MVC ของ Laravel ประกอบด้วย 4 ขั้นตอน ดังนี้

1) ส่วนกำหนดเส้นทาง รับคำขอ HTTP จากผู้ใช้งาน แล้วส่งต่อไปยังคอนโทรลเลอร์ที่รับผิดชอบตามเส้นทางที่กำหนดไว้

2) ส่วนควบคุม ดำเนินการประมวลผลตรรกะของแอปพลิเคชัน สื่อสารกับโมเดลเพื่ออ่านหรือแก้ไขข้อมูล และจัดเตรียมผลลัพธ์สำหรับการแสดงผล

3) ส่วนแบบจำลองข้อมูล เชื่อมโยงและจัดการข้อมูลในฐานข้อมูลเชิงสัมพันธ์ผ่าน Eloquent ORM ตามคำสั่งของคอนโทรลเลอร์

4) ส่วนแสดงผล รับข้อมูลจากคอนโทรลเลอร์มาแปลงให้อยู่ในรูปเอกสาร HTML หรือโครงสร้าง JSON แล้วส่งกลับไปยังผู้ใช้งานในรูปการตอบกลับ HTTP

2.5.1 ข้อดี

2.5.1.1 มีเครื่องมือพื้นฐานครบ ลดเวลาในการพัฒนา

2.5.1.2 โครงสร้างโค้ดเป็นระเบียบตามรูปแบบ MVC

2.5.1.3 มีเอกสารและชุมชนผู้ใช้งานจำนวนมาก

2.5.2 ข้อเสีย

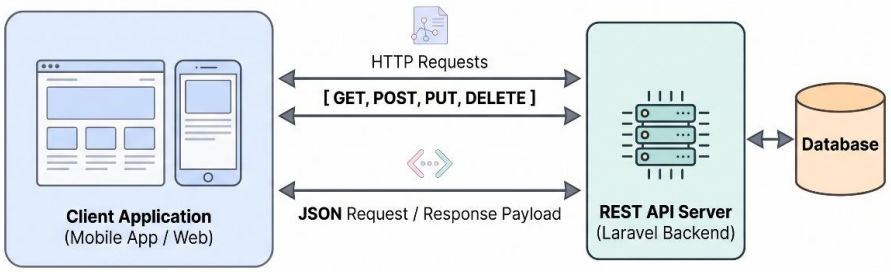
2.5.2.1 ใช้ทรัพยากรเครื่องแม่ข่ายมากกว่าการเขียน PHP แบบไม่ใช้เฟรมเวิร์ก

2.5.2.2 ผู้พัฒนาต้องเรียนรู้ข้อกำหนดของเฟรมเวิร์กก่อนใช้งาน

2.5.3 การนำไปใช้ในโครงการ ผู้จัดทำใช้ Laravel 10 พัฒนาระบบบริการหลังบ้านทั้งหมด ได้แก่ การรับ Webhook การส่งข้อความออกไปยังแพลตฟอร์ม คิวงาน แชทบอท การส่งการแจ้งเตือน และ REST API ที่แอปพลิเคชันเรียกใช้

2.6 REST API

REST API คือรูปแบบการออกแบบส่วนต่อประสานที่ใช้โพรโทคอล HTTP ในการสื่อสารระหว่างระบบ [6] โดยระบุทรัพยากรด้วย URL และใช้เมธอด GET POST PUT PATCH และ DELETE แทนการอ่าน สร้าง แก้ไข และลบข้อมูล ข้อมูลที่รับส่งนิยมใช้รูปแบบ JSON และเครื่องแม่ข่ายไม่เก็บสถานะของผู้เรียก คำขอแต่ละครั้งจึงต้องแนบโทเคนยืนยันตัวตนมาด้วย ดังรูปที่ 2.5



รูปที่ 2.5 การรับส่งข้อมูลผ่าน REST API

สถาปัตยกรรมการสื่อสารข้อมูลผ่าน REST API ในระบบประกอบด้วย 3 ส่วนสำคัญ ดังนี้

1) แอปพลิเคชันฝั่งผู้ใช้งาน ส่งคำขอผ่านเมธอดมาตรฐานของ HTTP ได้แก่ GET POST PUT และ DELETE ไปยังเครื่องแม่ข่ายบริการหลังบ้าน

2) โครงสร้างข้อมูล JSON ใช้เป็นสื่อกลางในการแลกเปลี่ยนข้อมูลระหว่างผู้เรียกและผู้ให้บริการ ทั้งส่วนข้อมูลนำเข้าและส่วนข้อมูลตอบกลับ

3) เครื่องแม่ข่ายบริการหลังบ้าน ตรวจสอบสิทธิ์ของคำขอ ติดต่อจัดการข้อมูลในฐานข้อมูล และแปลงผลลัพธ์ส่งกลับไปยังแอปพลิเคชัน

2.6.1 ข้อดี

2.6.1.1 ใช้มาตรฐาน HTTP ที่ทุกแพลตฟอร์มรองรับ

2.6.1.2 แยกการพัฒนาแอปพลิเคชันออกจากระบบหลังบ้านได้ชัดเจน

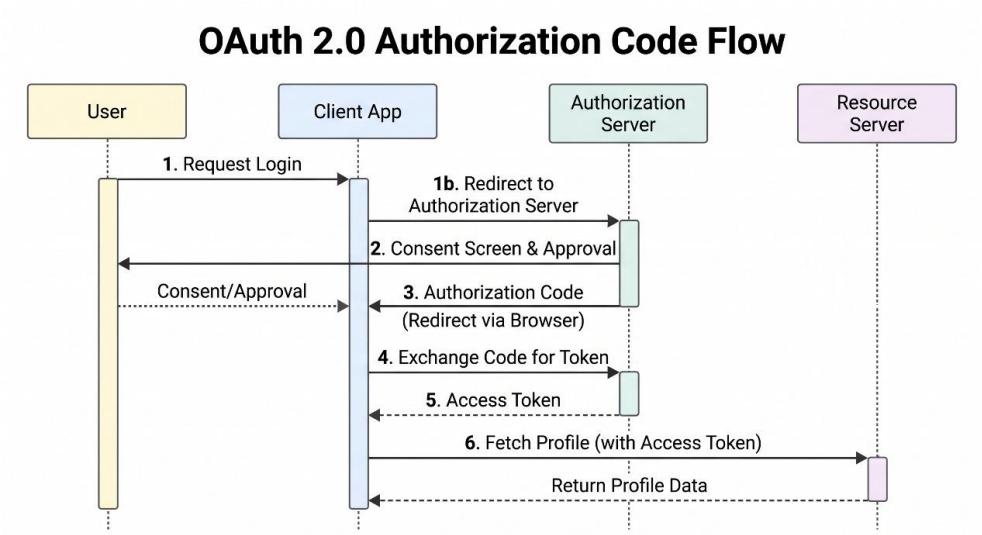
2.6.2 ข้อเสีย

2.6.2.1 หน้าจอที่ใช้ข้อมูลหลายชุดต้องเรียกหลายครั้ง

2.6.3 การนำไปใช้ในโครงการ ระบบหลังบ้านเปิด REST API ประมาณ 99 เส้นทาง ภายใต้ mobile/ และออกโทเคนด้วย Laravel Sanctum เมื่อเข้าสู่ระบบสำเร็จ แอปพลิเคชันใช้ flutter_secure_storage เก็บโทเคนไว้ในพื้นที่จัดเก็บที่เข้ารหัสของอุปกรณ์ และแนบโทเคนไปกับทุกคำขอในส่วนหัว Authorization หากระบบหลังบ้านตอบว่าโทเคนไม่ถูกต้อง แอปพลิเคชันจะลบโทเคนและกลับไปหน้าเข้าสู่ระบบ

2.7 OAuth 2.0

OAuth 2.0 คือมาตรฐานการอนุญาตให้แอปพลิเคชันเข้าถึงข้อมูลของผู้ใช้บนบริการอื่นโดยผู้ใช้ไม่ต้องให้รหัสผ่านแก่แอปพลิเคชัน [7] ผู้ใช้เข้าสู่ระบบและยินยอมที่ผู้ให้บริการ แล้วแอปพลิเคชันจะได้รับ Access Token สำหรับเรียกใช้ API แทนผู้ใช้ ดังรูปที่ 2.6



รูปที่ 2.6 ขั้นตอนการขอสิทธิ์ด้วย OAuth 2.0

ขั้นตอนการขอสิทธิ์และการยืนยันตัวตนตามมาตรฐาน OAuth 2.0 ประกอบด้วย 4 ขั้นตอนหลัก ดังนี้

1) การส่งคำขอเข้าสู่ระบบ ผู้ใช้งานส่งคำขอเข้าสู่ระบบผ่านแอปพลิเคชัน จากนั้นระบบจะเปลี่ยนเส้นทางไปยังเครื่องแม่ข่ายยืนยันสิทธิ์ของผู้ให้บริการภายนอก

2) การให้ความยินยอมและการรับรหัส เครื่องแม่ข่ายยืนยันสิทธิ์แสดงหน้าจอยินยอม เมื่อผู้ใช้ยินยอม ระบบจะส่งรหัส Authorization Code กลับมายังแอปพลิเคชัน

3) การแลกเปลี่ยนรหัสเป็นโทเคน แอปพลิเคชันส่ง Authorization Code ไปยังเครื่องแม่ข่ายยืนยันสิทธิ์เพื่อขอรับ Access Token สำหรับเข้าถึงข้อมูล

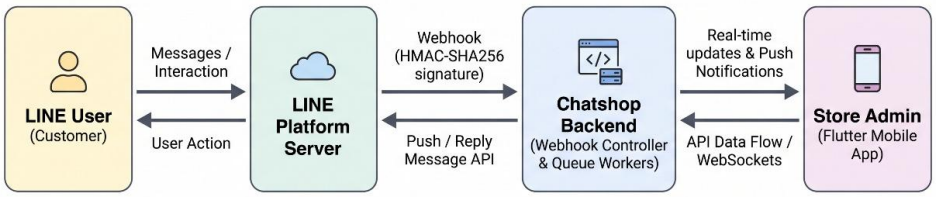
4) การดึงข้อมูลผู้ใช้งาน แอปพลิเคชันนำ Access Token ไปเรียกข้อมูลโปรไฟล์จากเครื่องแม่ข่ายทรัพยากรเพื่อนำมาประมวลผลในระบบ

2.7.1 การนำไปใช้ในโครงการ แอปพลิเคชันเรียกหน้าเข้าสู่ระบบของ Facebook และ Google บนสมาร์ตโฟนโดยตรง แล้วส่งโทเคนที่ได้ไปยังระบบหลังบ้าน ระบบหลังบ้านนำโทเคนไปตรวจสอบกับผู้ให้บริการ จากนั้นสร้างหรือค้นหาบัญชีผู้ใช้และออกโทเคนของระบบกลับมาให้แอปพลิเคชันใช้งาน นอกจากนี้ระบบหลังบ้านยังใช้ OAuth ของ Facebook ขอสิทธิ์จัดการข้อความของเพจร้านค้า

2.8 LINE Messaging API

LINE Messaging API คือบริการของ LINE สำหรับเชื่อมต่อระบบภายนอกกับบัญชี LINE Official Account [8] ระบบรับข้อความจากผู้ใช้ผ่าน Webhook และส่งข้อความกลับผ่าน API โดยยืนยันตัวตนด้วย Channel Access Token และตรวจสอบ Webhook ด้วย Channel Secret ดังรูปที่ 2.7

Figure 2.7: LINE Messaging API Architecture



รูปที่ 2.7 การรับส่งข้อความผ่าน LINE Messaging API

การเชื่อมต่อและแลกเปลี่ยนข้อความผ่าน LINE Messaging API ประกอบด้วย 4 องค์ประกอบสำคัญ ดังนี้

1) ลูกค้าผู้ใช้งาน LINE ส่งข้อความสนทนาหรือดำเนินการต่างๆ ผ่านบัญชีทางการของร้านค้า

2) เครื่องแม่ข่ายแพลตฟอร์ม LINE รับข้อมูลจากลูกค้าแล้วส่งต่อมายังระบบบริการหลังบ้านผ่าน Webhook พร้อมแนบลายเซ็นดิจิทัล HMAC-SHA256

3) ระบบบริการหลังบ้านของระบบ ตรวจสอบความถูกต้องของลายเซ็น นำข้อความเข้าสู่คิวงาน และส่งการแจ้งเตือนแบบเรียลไทม์ไปยังเจ้าหน้าที่

4) แอปพลิเคชันของเจ้าหน้าที่ร้านค้า รับข้อความและเปิดอ่านผ่านแอปพลิเคชันบนสมาร์ตโฟน โดยการส่งข้อความตอบกลับจะเรียกใช้ Push API ไปยังแพลตฟอร์ม LINE

2.8.1 ข้อดี

2.8.1.1 เข้าถึงผู้ใช้ LINE ซึ่งเป็นแอปพลิเคชันแชทที่คนไทยใช้มากที่สุด

2.8.1.2 ส่งข้อความได้หลายรูปแบบ เช่น ข้อความ รูปภาพ และสติกเกอร์

2.8.2 ข้อเสีย

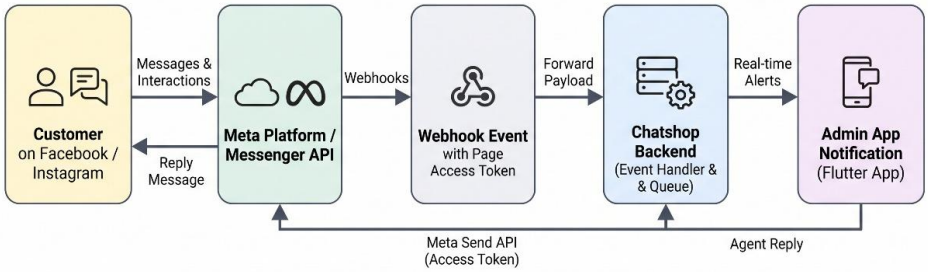
2.8.2.1 ไฟล์สื่อจากผู้ใช้ไม่มี URL สาธารณะ ต้องดาวน์โหลดผ่าน API

2.8.3 การนำไปใช้ในโครงการ ระบบหลังบ้านรับข้อความจาก LINE ผ่าน Webhook ของแต่ละช่องทาง และส่งข้อความกลับด้วย Push API โดยแสดงสถานะกำลังโหลดให้ลูกค้าเห็นระหว่างที่แชทบอทสร้างคำตอบ

2.9 Meta Graph API

Meta Graph API คือ API ที่บริษัท Meta เปิดให้นักพัฒนาใช้เข้าถึงข้อมูลและบริการบนแพลตฟอร์ม Facebook และ Instagram [9] โครงงานนี้ใช้ส่วน Messenger Platform สำหรับรับส่งข้อความกับผู้ใช้ผ่านเพจ Facebook และบัญชี Instagram โดยใช้ Page Access Token ในการยืนยันตัวตน ดังรูปที่ 2.8

Figure 2.8: Meta Messenger & Instagram Direct Integration



รูปที่ 2.8 การรับส่งข้อความผ่าน Messenger Platform

กระบวนการรับส่งข้อความผ่าน Messenger Platform ของ Meta ประกอบด้วย 4 ขั้นตอนสำคัญ ดังนี้

1) ลูกค้าส่งข้อความสนทนาผ่านเพจ Facebook หรือบัญชี Instagram

2) แพลตฟอร์ม Meta รับข้อความและส่งต่อข้อมูลเหตุการณ์ผ่าน Webhook พร้อมระบุ Page Access Token มายังเครื่องแม่ข่าย

3) ระบบบริการหลังบ้าน รับข้อมูลผ่านตัวจัดการเหตุการณ์ นำเข้าคิวงานประมวลผล และส่งการแจ้งเตือนแบบเรียลไทม์ไปยังแอปพลิเคชัน

4) เจ้าหน้าที่ตอบกลับข้อความจากแอปพลิเคชัน โดยระบบบริการหลังบ้านจะส่งข้อความออกไปยังแพลตฟอร์ม Meta ผ่าน Meta Send API

2.9.1 ข้อดี

2.9.1.1 ใช้ API ชุดเดียวส่งข้อความได้ทั้ง Messenger และ Instagram

2.9.1.2 ดึงประวัติบทสนทนาเดิมของเพจมาเก็บในระบบได้

2.9.2 ข้อเสีย

2.9.2.1 ข้อความอัตโนมัติส่งได้ภายใน 24 ชั่วโมงหลังข้อความล่าสุดเท่านั้น

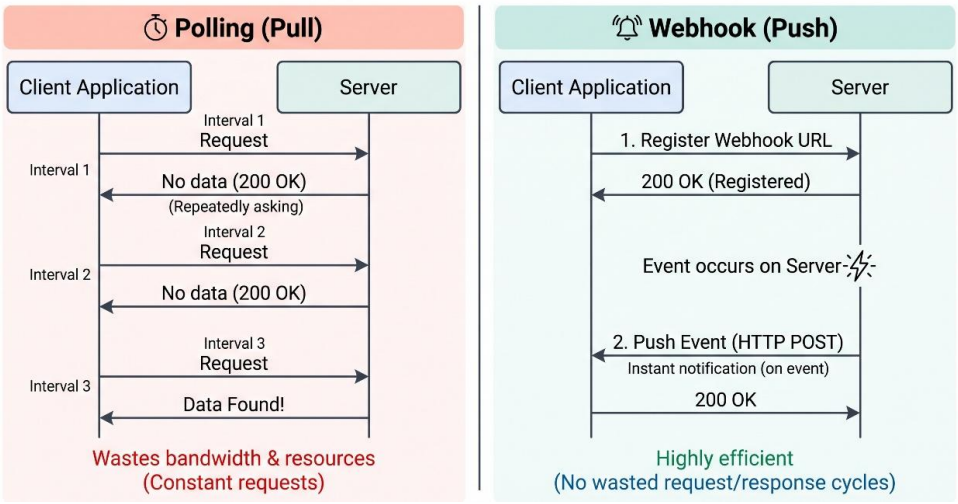
2.9.2.2 โทเคนของเพจหมดอายุหรือถูกยกเลิกได้ ระบบต้องตรวจจับข้อผิดพลาดนี้ ปิดการเชื่อมต่อช่องทางนั้น และแจ้งให้ร้านค้าเชื่อมต่อใหม่

2.9.3 การนำไปใช้ในโครงการ ระบบหลังบ้านใช้ Graph API เวอร์ชัน 19.0 ส่งข้อความตอบกลับ ดึงรูปโปรไฟล์ลูกค้า และนำเข้าประวัติแชทย้อนหลังได้ไม่เกิน 90 วัน

2.10 Webhook

Webhook คือกลไกที่ระบบต้นทางส่งคำขอ HTTP ไปยัง URL ที่ระบบปลายทางได้ลงทะเบียนไว้ล่วงหน้าทันทีเมื่อเกิดเหตุการณ์ [10] เช่น มีข้อความใหม่ ระบบปลายทางจึงรับข้อมูลได้ทันทีโดยไม่ต้องสอบถามซ้ำเป็นระยะ ดังรูปที่ 2.9 ผู้ให้บริการมักแนบลายเซ็นดิจิทัลที่สร้างจาก HMAC มาด้วย เพื่อให้ระบบปลายทางตรวจสอบได้ว่าข้อมูลมาจากผู้ให้บริการจริงและไม่ถูกแก้ไขระหว่างทาง

Figure 2.9: Polling vs Webhook Architecture



รูปที่ 2.9 การทำงานของ Webhook เทียบกับการสอบถามข้อมูลเป็นระยะ

ข้อเปรียบเทียบระหว่างสถาปัตยกรรมแบบ Polling และ Webhook มีสาระสำคัญ 2 รูปแบบ ดังนี้

1) รูปแบบ Polling แอปพลิเคชันต้องส่งคำขอไปยังเครื่องแม่ข่ายซ้ำๆ เป็นระยะ แม้ไม่มีข้อมูลใหม่ ทำให้สิ้นเปลืองทรัพยากรการประมวลผลและแบนด์วิดท์ของระบบ

2) รูปแบบ Webhook แอปพลิเคชันลงทะเบียน URL รับข้อมูลไว้ล่วงหน้า เมื่อเกิดเหตุการณ์บนเครื่องแม่ข่าย ระบบจะส่งข้อมูลผ่านคำขอ HTTP POST ในทันที ทำให้มีความรวดเร็วและใช้ทรัพยากรได้อย่างมีประสิทธิภาพสูงสุด

2.10.1 ข้อดี

2.10.1.1 รับข้อความใหม่ได้ทันที ไม่ต้องสอบถามข้อมูลซ้ำเป็นระยะ

2.10.1.2 ลดภาระของเครื่องแม่ข่ายทั้งฝั่งผู้ให้บริการและฝั่งระบบ

2.10.2 ข้อเสีย

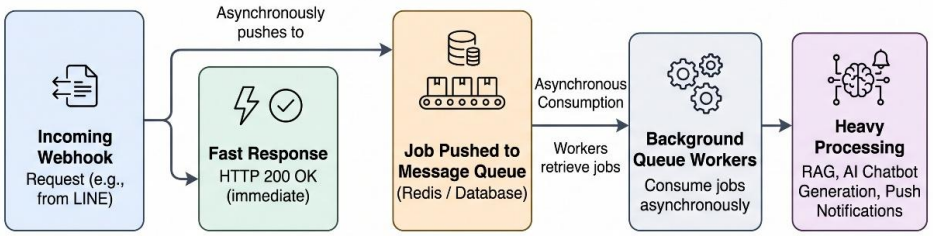
2.10.2.1 ระบบต้องตอบกลับให้เร็ว หากตอบช้าผู้ให้บริการจะส่งซ้ำ

2.10.3 การนำไปใช้ในโครงการ ระบบหลังบ้านตรวจสอบลายเซ็น X-Line-Signature ของ LINE และ X-Hub-Signature-256 ของ Meta ด้วย HMAC-SHA256 จากนั้นส่งข้อมูลเข้าคิวงานทันที เพื่อให้ตอบกลับผู้ให้บริการได้รวดเร็ว แล้วจึงบันทึกข้อความและเรียกแชทบอทในเบื้องหลัง

2.11 Queue และ Task Scheduling

Queue คือการย้ายงานที่ใช้เวลานานออกไปทำงานเบื้องหลัง ทำให้ระบบตอบกลับคำขอได้ทันที ดังรูปที่ 2.10 งานแต่ละประเภทแยกไว้คนละคิวได้เพื่อกำหนดลำดับความสำคัญ ส่วน Task Scheduling คือการกำหนดให้คำสั่งทำงานอัตโนมัติตามรอบเวลา [11]

Figure 2.10: Asynchronous Job Queue Architecture



รูปที่ 2.10 การทำงานของคิวงาน

สถาปัตยกรรมการทำงานของคิวงานแบบไม่ประสานเวลาประกอบด้วย 4 ส่วนสำคัญ ดังนี้

1) การตอบกลับคำขอขาเข้า เมื่อได้รับข้อมูลคำขอจาก Webhook ระบบจะตอบกลับรหัสสถานะ 200 ในทันที เพื่อป้องกันไม่ให้ระบบภายนอกรอนานจนส่งข้อความซ้ำ

2) การบรรจุงานลงในคิว ระบบสร้างภารกิจงานแล้วนำเข้าสู่คิวงานในฐานข้อมูล เพื่อรอให้ตัวประมวลผลนำไปดำเนินการต่อไป

3) ตัวประมวลผลคิวงาน ทำหน้าที่ดึงภารกิจงานที่รออยู่ในคิวมาประมวลผลในเบื้องหลังตามลำดับความสำคัญ โดยไม่รบกวนการทำงานหลักของเครื่องแม่ข่าย

4) การประมวลผลภารกิจที่ใช้เวลานาน รองรับงานที่มีความซับซ้อนสูง เช่น การค้นคืนข้อมูล RAG การสร้างคำตอบของแชทบอท และการส่งการแจ้งเตือนไปยังอุปกรณ์ปลายทาง

2.11.1 การนำไปใช้ในโครงการ

2.11.1.1 คิว high ใช้ประมวลผล Webhook ขาเข้าและการนำเข้าประวัติแชท

2.11.1.2 คิว default ใช้สร้างคำตอบของแชทบอท สรุปบทสนทนา จัดทำดัชนีเอกสารความรู้ และส่งบรอดแคสต์เป็นชุด ชุดละ 100 บทสนทนา

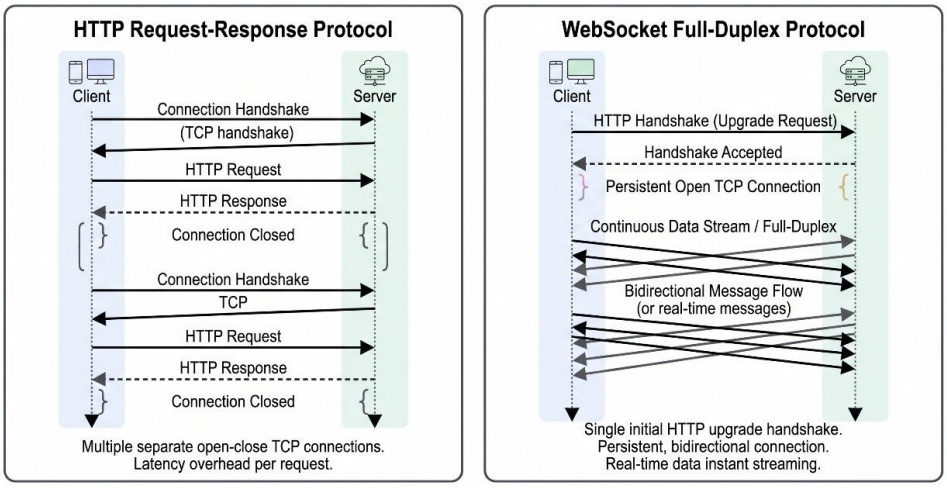
2.11.1.3 คิว typing ใช้ส่งสถานะกำลังพิมพ์ระหว่างที่แชทบอทสร้างคำตอบ

2.11.1.4 ตัวจัดตารางเวลาตรวจหาแชทที่ค้างตอบนานเกินกำหนดทุก 1 นาที

2.12 WebSocket และ Laravel Reverb

WebSocket คือโพรโทคอลที่เปิดการเชื่อมต่อค้างไว้ระหว่างผู้ใช้กับเครื่องแม่ข่าย ทำให้เครื่องแม่ข่ายส่งข้อมูลไปยังผู้ใช้ได้ทันทีโดยผู้ใช้ไม่ต้องส่งคำขอซ้ำ ดังรูปที่ 2.11 ส่วน Laravel Reverb คือเซิร์ฟเวอร์ WebSocket ของ Laravel [12] ที่ใช้โพรโทคอลเดียวกับ Pusher และรองรับช่องส่วนตัวที่ต้องขออนุญาตจากระบบหลังบ้านก่อนรับข้อมูล

Figure 2.11: HTTP vs WebSocket Communication Protocols



รูปที่ 2.11 การเชื่อมต่อแบบ WebSocket เทียบกับ HTTP

การเปรียบเทียบโพรโทคอลการสื่อสารระหว่าง HTTP และ WebSocket มีคุณลักษณะ 2 รูปแบบ ดังนี้

1) โพรโทคอล HTTP ทำงานแบบส่งคำขอและรอรับคำตอบ ต้องสร้างและตัดการเชื่อมต่อ TCP ใหม่ในทุกรอบการสื่อสาร ทำให้เกิดความล่าช้าสะสม

2) โพรโทคอล WebSocket ทำการจับมือผ่าน HTTP เพียงครั้งแรกเพื่อยกระดับการเชื่อมต่อ จากนั้นเปิดช่องทาง TCP ค้างไว้เพื่อแลกเปลี่ยนข้อมูลสองทิศทางแบบเรียลไทม์ได้อย่างต่อเนื่อง

2.12.1 ข้อดี

2.12.1.1 แสดงข้อความใหม่ได้ทันทีโดยไม่ต้องดึงข้อมูลซ้ำเป็นระยะ

2.12.1.2 ช่องส่วนตัวป้องกันไม่ให้ผู้ใช้ร้านอื่นรับข้อมูลของร้านค้าได้

2.12.2 ข้อเสีย

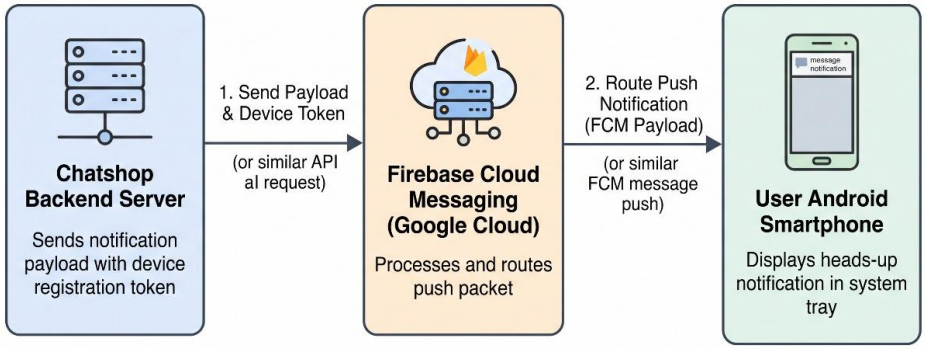
2.12.2.1 การเชื่อมต่อหลุดได้เมื่อสัญญาณเครือข่ายของสมาร์ตโฟนไม่เสถียร

2.12.3 การนำไปใช้ในโครงการ ระบบหลังบ้านส่งเหตุการณ์ของร้านค้า เช่น ข้อความใหม่ การแจ้งเตือน และคำสั่งซื้อที่เปลี่ยนสถานะ ผ่านช่องส่วนตัวของแต่ละร้านค้า แอปพลิเคชันใช้แพ็กเกจ laravel_reverb เชื่อมต่อช่องนี้หลังเข้าสู่ระบบ แล้วปรับรายการบทสนทนาและหน้าต่างสนทนาที่เปิดอยู่ทันทีเมื่อได้รับเหตุการณ์

2.13 Firebase Cloud Messaging (FCM)

Firebase Cloud Messaging คือบริการส่ง Push Notification ที่ให้บริการโดย Google [13] เครื่องแม่ข่ายส่งการแจ้งเตือนไปยังอุปกรณ์ผ่านโทเคนประจำเครื่องที่แอปพลิเคชันลงทะเบียนไว้ ดังรูปที่ 2.12 ผู้ใช้จึงได้รับการแจ้งเตือนแม้ไม่ได้เปิดแอปพลิเคชันอยู่ และแตะการแจ้งเตือนเพื่อเปิดหน้าจอที่เกี่ยวข้องได้

Figure 2.12: FCM Push Notification Delivery Flow



รูปที่ 2.12 การส่ง Push Notification ผ่าน Firebase Cloud Messaging

กระบวนการกระจายการแจ้งเตือนผ่าน Firebase Cloud Messaging ประกอบด้วย 3 ขั้นตอน ดังนี้

1) การส่งข้อมูลจากระบบหลังบ้าน เครื่องแม่ข่ายของระบบรวบรวมข้อมูลการแจ้งเตือนและแนบโทเคนประจำเครื่องส่งไปยังบริการ FCM บนระบบคลาวด์ของ Google

2) การประมวลผลและส่งต่อ บริการ FCM ตรวจสอบความถูกต้องและจัดส่งแพ็กเกจข้อมูลไปยังสมาร์ตโฟนปลายทางที่มีโทเคนตรงกัน

3) การแสดงผลบนอุปกรณ์ สมาร์ตโฟนของผู้ใช้งานรับข้อมูลและแสดงแถบการแจ้งเตือนบนถาดแจ้งเตือนของระบบปฏิบัติการ แม้แอปพลิเคชันจะปิดอยู่

2.13.1 ข้อดี

2.13.1.1 ใช้งานได้โดยไม่มีค่าใช้จ่าย

2.13.1.2 ส่งการแจ้งเตือนถึงอุปกรณ์ได้แม้แอปพลิเคชันถูกปิดอยู่

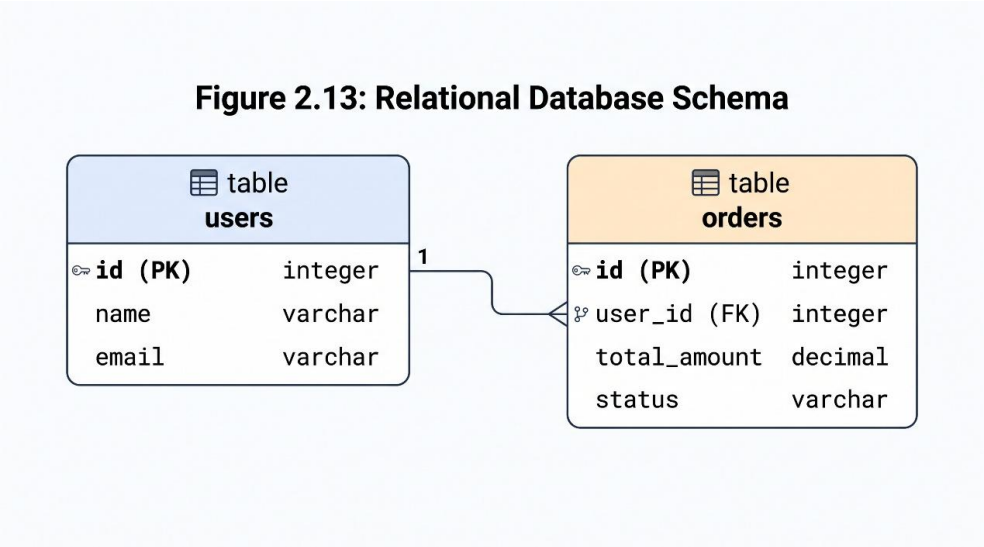
2.13.2 ข้อเสีย

2.13.2.1 ขณะแอปพลิเคชันเปิดอยู่ ระบบปฏิบัติการจะไม่แสดงการแจ้งเตือนให้เอง แอปพลิเคชันต้องจัดการแสดงผลเอง

2.13.3 การนำไปใช้ในโครงการ แอปพลิเคชันลงทะเบียนโทเคนของอุปกรณ์เมื่อเข้าสู่ระบบ ปรับปรุงโทเคนเมื่อมีการเปลี่ยนแปลง และยกเลิกเมื่อออกจากระบบ ส่วนระบบหลังบ้านจะส่งการแจ้งเตือนเมื่อมีข้อความใหม่ แชทค้างตอบนานเกินกำหนด หรือแชทบอทสร้างคำตอบไม่สำเร็จ หากแอปพลิเคชันเปิดอยู่จะแสดงการแจ้งเตือนด้วยแพ็กเกจ flutter_local_notifications แทน เมื่อผู้ใช้แตะการแจ้งเตือน แอปพลิเคชันจะเปิดหน้าต่างสนทนานั้นทันที

2.14 MySQL

MySQL คือระบบจัดการฐานข้อมูลเชิงสัมพันธ์แบบโอเพนซอร์ส [14] จัดเก็บข้อมูลในรูปตารางที่ประกอบด้วยแถวและคอลัมน์ แต่ละตารางมีคีย์หลักสำหรับระบุแถวที่ไม่ซ้ำกัน และใช้คีย์นอกอ้างอิงแถวของตารางอื่นเพื่อบังคับความสัมพันธ์ของข้อมูล ดังรูปที่ 2.13 ผู้ใช้จัดการข้อมูลด้วยภาษา SQL ระบบรองรับธุรกรรมตามคุณสมบัติ ACID ที่ทำให้กลุ่มคำสั่งสำเร็จทั้งหมดหรือไม่สำเร็จเลย และสร้างดัชนีเพื่อเพิ่มความเร็วในการค้นหาได้ นอกจากนี้ MySQL ยังมีชนิดข้อมูล JSON สำหรับเก็บข้อมูลที่เป็นรายการไว้ในคอลัมน์เดียว



รูปที่ 2.13 ตัวอย่างตารางที่เชื่อมกันด้วยคีย์หลักและคีย์นอก

โครงสร้างความสัมพันธ์ระหว่างตารางในฐานข้อมูลเชิงสัมพันธ์ประกอบด้วย 2 ตาราง หลัก ดังนี้

1) ตารางข้อมูลผู้ใช้งาน ทำหน้าที่เก็บข้อมูลพื้นฐาน เช่น รหัสประจำตัวผู้ใช้ซึ่งเป็นคีย์หลัก ชื่อผู้ใช้ และที่อยู่อีเมล

2) ตารางข้อมูลคำสั่งซื้อ ทำหน้าที่เก็บรายการสั่งซื้อ โดยมีรหัสคำสั่งซื้อเป็นคีย์หลัก และเก็บรหัสประจำตัวผู้ใช้เป็นคีย์นอก เพื่อเชื่อมโยงความสัมพันธ์แบบหนึ่งต่อกลุ่มเข้ากับตารางผู้ใช้งาน

2.14.1 ข้อดี

2.14.1.1 รักษาความถูกต้องของข้อมูลที่สัมพันธ์กันด้วยคีย์หลักและคีย์นอก

2.14.1.2 รองรับธุรกรรม เหมาะกับข้อมูลที่ต้องการความถูกต้องสูง

2.14.1.3 รวมข้อมูลจากหลายตารางในคำสั่งเดียวได้ด้วยการเชื่อมตาราง

2.14.2 ข้อเสีย

2.14.2.1 ต้องกำหนดโครงสร้างตารางล่วงหน้า ไม่เหมาะกับข้อมูลที่เปลี่ยนบ่อย

2.14.2.2 การเพิ่มคอลัมน์ในตารางที่มีข้อมูลจำนวนมากใช้เวลานาน

2.14.3 การนำไปใช้ในโครงการ ผู้จัดทำใช้ MySQL จัดเก็บข้อมูลที่มีความสัมพันธ์กันชัดเจนและต้องการความถูกต้องสูง ดังนี้

2.14.3.1 ข้อมูลร้านค้าและผู้ใช้ ได้แก่ ร้านค้า สมาชิก บทบาท และคำเชิญ

2.14.3.2 ข้อมูลธุรกิจและช่องทาง ได้แก่ ธุรกิจ ช่องทางที่เชื่อมต่อ และการตั้งค่าแชทบอทของแต่ละธุรกิจ

2.14.3.3 ข้อมูลการขาย ได้แก่ ลูกค้า สินค้า และคำสั่งซื้อ

2.14.3.4 ข้อมูลสนับสนุนการตอบแชท ได้แก่ แท็ก ข้อความสำเร็จรูป กฎตอบกลับอัตโนมัติ ผู้รับผิดชอบบทสนทนา และประวัติการนำเข้าแชท

2.14.4 แนวทางการออกแบบตารางในโครงการ ดังนี้

2.14.4.1 ทุกตารางที่เป็นข้อมูลของร้านค้าอ้างอิงตารางร้านค้าด้วยคีย์นอกแบบลบตามกัน เมื่อร้านค้าถูกลบ ข้อมูลที่เกี่ยวข้องจะถูกลบตามโดยอัตโนมัติ

2.14.4.2 เก็บข้อมูลที่เป็นรายการ เช่น สิทธิ์ของบทบาทและคีย์เวิร์ดของกฎตอบกลับ ไว้ในคอลัมน์ชนิด JSON

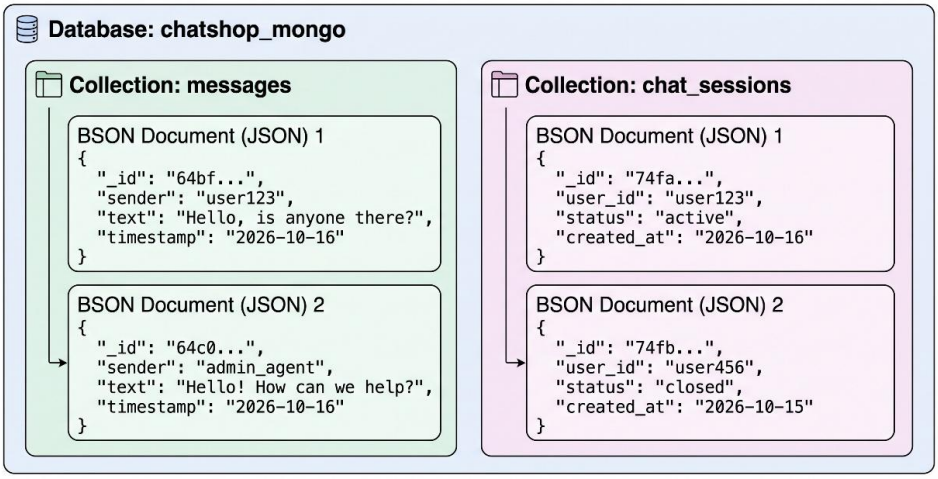
2.14.4.3 ใช้ชุดอักขระ utf8mb4 ซึ่งรองรับทั้งภาษาไทยและอีโมจิ

2.14.4.4 สร้างและแก้ไขโครงสร้างตารางผ่าน Migration ของ Laravel จึงติดตามประวัติการเปลี่ยนแปลงโครงสร้างได้

2.15 MongoDB

MongoDB คือฐานข้อมูลแบบเอกสาร จัดเก็บข้อมูลเป็นเอกสารรูปแบบ BSON ซึ่งมีโครงสร้างคล้าย JSON [15] และรวมเอกสารประเภทเดียวกันไว้ในคอลเลกชัน เอกสารในคอลเลกชันเดียวกันไม่จำเป็นต้องมีฟิลด์เหมือนกัน และเก็บอาร์เรย์หรือเอกสารย่อยไว้ในเอกสารได้โดยตรง ดังรูปที่ 2.14 เอกสารทุกฉบับมีรหัส ObjectId ที่ระบบสร้างให้เป็นคีย์หลัก การใช้ฐานข้อมูลหลายประเภทในระบบเดียวตามลักษณะของข้อมูลเรียกว่า Polyglot Persistence [16]

Figure 2.14: MongoDB NoSQL Database Architecture



รูปที่ 2.14 โครงสร้างฐานข้อมูล คอลเลกชัน และเอกสารของ MongoDB

สถาปัตยกรรมการจัดเก็บข้อมูลของ MongoDB ในระบบประกอบด้วย 2 คอลเลกชันหลัก ดังนี้

1) คอลเลกชันข้อความ ทำหน้าที่เก็บเอกสารข้อความสนทนาในรูปแบบ BSON ซึ่งประกอบด้วยรหัสเอกสาร รหัสผู้ส่ง ข้อความ และเวลาที่บันทึกข้อมูล

2) คอลเลกชันรอบการสนทนา ทำหน้าที่เก็บเอกสารบทสนทนาประกอบด้วยรหัสเอกสาร รหัสผู้ใช้งาน สถานะการสนทนา และเวลาที่เริ่มสร้างบทสนทนา

2.15.1 ข้อดี

2.15.1.1 โครงสร้างข้อมูลยืดหยุ่น เพิ่มฟิลด์ได้โดยไม่ต้องแก้โครงสร้างตาราง

2.15.1.2 รองรับข้อมูลปริมาณมากและเก็บอาร์เรย์ในเอกสารได้โดยตรง

2.15.1.3 อ่านข้อมูลที่เกี่ยวข้องได้จากเอกสารเดียวโดยไม่ต้องเชื่อมหลายตาราง

2.15.2 ข้อเสีย

2.15.2.1 ไม่มีการบังคับความสัมพันธ์ระหว่างข้อมูลเหมือนฐานข้อมูลเชิงสัมพันธ์

2.15.2.2 ผู้พัฒนาต้องตรวจสอบความถูกต้องของฟิลด์ในโปรแกรมเอง

2.15.3 การนำไปใช้ในโครงการ ผู้จัดทำใช้ MongoDB เก็บข้อมูลที่มีปริมาณมาก เพิ่มขึ้นตลอดเวลา หรือมีรูปแบบไม่แน่นอน แบ่งเป็น 5 คอลเลกชัน ดังนี้

2.15.3.1 conversations เก็บบทสนทนาของลูกค้าแต่ละรายในแต่ละช่องทาง พร้อมแท็ก ผู้รับผิดชอบ และโหมดแชทบอท

2.15.3.2 messages เก็บข้อความทุกข้อความ ซึ่งมีฟิลด์ต่างกันตามประเภท เช่น ข้อความสติกเกอร์มีรหัสสติกเกอร์ และข้อความไฟล์มีชื่อและขนาดไฟล์

2.15.3.3 knowledge_documents เก็บเอกสารความรู้ของแชทบอทแต่ละธุรกิจ

2.15.3.4 knowledge_chunks เก็บส่วนย่อยของเอกสารพร้อมเวกเตอร์

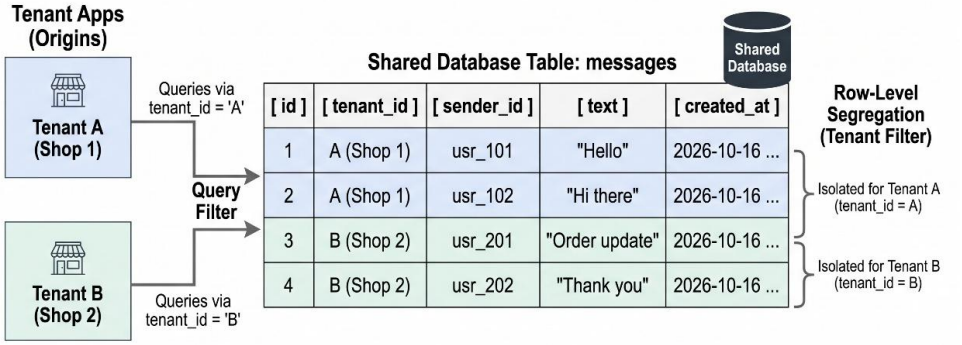
2.15.3.5 device_tokens เก็บโทเคนของอุปกรณ์สำหรับส่งการแจ้งเตือน

2.15.4 การเชื่อมต่อกับระบบหลังบ้าน ระบบหลังบ้านเชื่อมต่อ MongoDB ผ่านไลบรารี laravel-mongodb ซึ่งใช้ Model รูปแบบเดียวกับ Eloquent และเชื่อมโยงข้อมูลกับ MySQL ด้วยรหัสอ้างอิง เช่น เอกสารบทสนทนาเก็บรหัสร้านค้าและรหัสลูกค้าจาก MySQL ส่วนตารางคำสั่งซื้อเก็บรหัสเอกสารบทสนทนาจาก MongoDB

2.16 สถาปัตยกรรม Multi-Tenant

สถาปัตยกรรม Multi-Tenant คือการออกแบบให้ระบบเดียวให้บริการผู้เช่าหลายรายโดยข้อมูลของผู้เช่าแต่ละรายแยกออกจากกัน [17] วิธีที่ใช้กันทั่วไปคือใช้ฐานข้อมูลร่วมกันและเพิ่มคอลัมน์รหัสผู้เช่าในทุกตาราง แล้วกรองข้อมูลตามรหัสผู้เช่าทุกครั้งที่อ่านหรือเขียน ดังรูปที่ 2.15

Figure 2.15: Multi-Tenant Data Isolation



รูปที่ 2.15 การแยกข้อมูลของผู้เช่าด้วยคอลัมน์รหัสผู้เช่า

กลไกการแยกข้อมูลของผู้เช่าในระดับแถวข้อมูลประกอบด้วย 3 องค์ประกอบหลัก ดังนี้

1) แอปพลิเคชันของผู้เช่า แต่ละร้านค้าส่งคำขอเข้าใช้งานระบบพร้อมระบุรหัสประจำตัวผู้เช่าในคำขอ

2) ตัวกรองคำสอบถามข้อมูล ระบบบริการหลังบ้านนำรหัสผู้เช่ามาใช้เป็นเงื่อนไขในการสอบถามข้อมูล เพื่อจำกัดการเข้าถึงเฉพาะข้อมูลที่เป็นของร้านค้านั้น

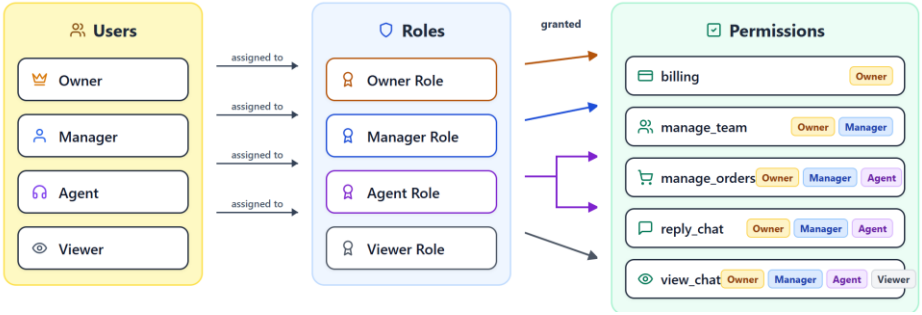
3) ตารางข้อมูลส่วนกลาง ฐานข้อมูลจัดเก็บข้อมูลของทุกร้านค้าไว้ในตารางเดียวกันโดยมีคอลัมน์รหัสผู้เช่ากำกับไว้ทุกแถว ทำให้เกิดการแยกข้อมูลอย่างเป็นอิสระและปลอดภัย

2.16.1 การนำไปใช้ในโครงการ ทุกตารางและทุกเอกสารมีรหัสร้านค้ากำกับ ร้านค้าหนึ่งแห่งมีได้หลายธุรกิจ แต่ละธุรกิจมีช่องทาง แบรนด์ และการตั้งค่าแชทบอทของตนเอง

2.17 การควบคุมสิทธิ์ตามบทบาท (RBAC)

RBAC (Role-Based Access Control) คือการควบคุมการเข้าถึงระบบโดยกำหนดสิทธิ์ให้แก่บทบาท แล้วมอบบทบาทให้ผู้ใช้ [18] ดังรูปที่ 2.16 เมื่อต้องการเปลี่ยนสิทธิ์ของผู้ใช้หลายคน ผู้ดูแลแก้ไขเฉพาะบทบาทโดยไม่ต้องแก้ทีละคน ช่วยลดความผิดพลาดในการกำหนดสิทธิ์

Figure 2.16: Role-Based Access Control (RBAC)



รูปที่ 2.16 ความสัมพันธ์ระหว่างผู้ใช้ บทบาท และสิทธิ์

โครงสร้างการควบคุมสิทธิ์ตามบทบาทในระบบแบ่งออกเป็น 3 ระดับสำคัญ ดังนี้

1) ผู้ใช้งาน ประกอบด้วยบุคลากรระดับต่างๆ ได้แก่ เจ้าของร้าน ผู้จัดการ เจ้าหน้าที่ และผู้ดูข้อมูล

2) บทบาท ทำหน้าที่เป็นกลุ่มของสิทธิ์ที่กำหนดขึ้นเพื่อมอบหมายให้แก่ผู้ใช้งาน โดยผู้ใช้งานแต่ละรายจะได้รับบทบาทที่เหมาะสมกับหน้าที่

3) สิทธิ์การเข้าถึง กำหนดขอบเขตการทำงานเฉพาะส่วน ได้แก่ การจัดการค่าบริการ การจัดการทีม การจัดการคำสั่งซื้อ การตอบข้อความ และการดูข้อความ ซึ่งแต่ละบทบาทจะได้รับชุดสิทธิ์ที่แตกต่างกันตามระดับความรับผิดชอบ

2.17.1 การนำไปใช้ในโครงการ ระบบมีบทบาทพื้นฐานที่กำหนดไว้ล่วงหน้า 5 บทบาท ได้แก่ owner, manager, admin, agent และ viewer และเจ้าของร้านสร้างบทบาทใหม่ได้ โดยเลือกจากสิทธิ์ทั้งหมด 12 รายการ เช่น การตอบข้อความ การมอบหมายงาน และการจัดการคำสั่งซื้อ โดยระบบหลังบ้านตรวจสอบสิทธิ์ทุกครั้งที่มีคำขอ

2.18 แชทบอท

แชทบอทคือโปรแกรมที่สนทนากับผู้ใช้ด้วยภาษาธรรมชาติผ่านช่องทางแชท [19] แชทบอทแบบกฎจะตอบตามคำสำคัญหรือเงื่อนไขที่กำหนดไว้ล่วงหน้า ส่วนแชทบอทที่ใช้แบบจำลองภาษาสามารถสร้างคำตอบใหม่ได้ตามบริบทของการสนทนา แต่อาจสร้างข้อมูลที่ไม่มีอยู่จริงได้ ร้านค้าจึงต้องให้แชทบอทส่งต่อบทสนทนาให้เจ้าหน้าที่เมื่อตอบคำถามไม่ได้

2.18.1 การนำไปใช้ในโครงการ แชทบอทตอบลูกค้าได้ทุกช่องทาง ร้านค้ากำหนดโหมดเริ่มต้นของแต่ละธุรกิจ และเจ้าหน้าที่เปลี่ยนโหมดรายบทสนทนาได้จากแอปพลิเคชัน ดังนี้

2.18.1.1 Manual เจ้าหน้าที่ตอบลูกค้าเองทั้งหมด

2.18.1.2 Suggest แชทบอทเสนอคำตอบให้เจ้าหน้าที่เลือกใช้

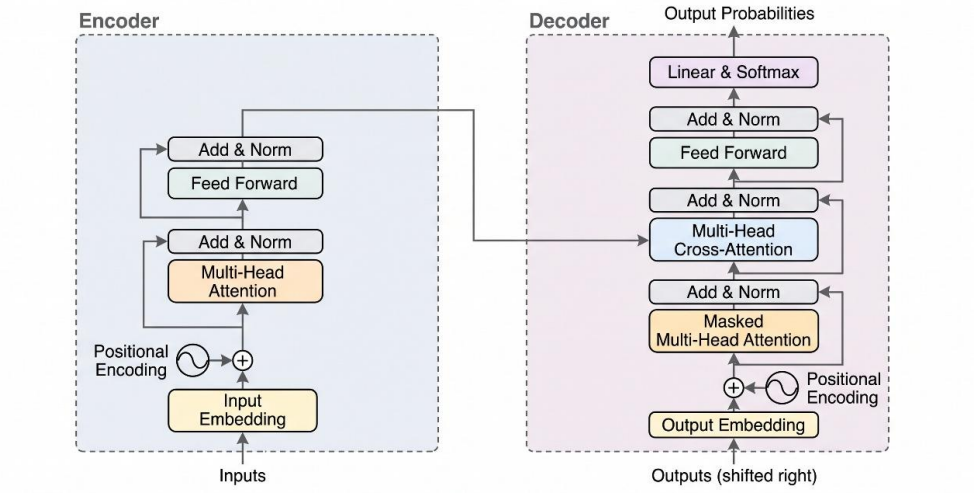
2.18.1.3 Autopilot แชทบอทตอบลูกค้าเอง ส่งต่อให้เจ้าหน้าที่เมื่อตอบไม่ได้

2.18.2 การตอบอัตโนมัติด้วยคีย์เวิร์ด นอกจากแชทบอทที่ใช้แบบจำลองภาษา ระบบมีการตอบอัตโนมัติแบบกฎ ร้านค้ากำหนดคีย์เวิร์ดและข้อความตอบกลับได้เอง เมื่อข้อความของลูกค้าตรงกับคีย์เวิร์ดทั้งข้อความหรือมีคีย์เวิร์ดอยู่ ระบบจะส่งข้อความที่กำหนดไว้ทันที

2.19 แบบจำลองภาษาขนาดใหญ่ (Large Language Model: LLM)

แบบจำลองภาษาขนาดใหญ่คือแบบจำลองที่ผ่านการฝึกด้วยข้อความจำนวนมหาศาลเพื่อทำนายคำถัดไปของข้อความ แบบจำลองส่วนใหญ่ใช้สถาปัตยกรรม Transformer ซึ่งใช้กลไก Self-Attention พิจารณาความสัมพันธ์ระหว่างคำทุกคำในข้อความพร้อมกัน [20] ดังรูปที่ 2.17 เมื่อแบบจำลองมีขนาดใหญ่เพียงพอ จะสามารถทำงานใหม่ได้จากคำสั่งและตัวอย่างที่ให้ไว้ในข้อความนำเข้า โดยไม่ต้องฝึกแบบจำลองเพิ่มเติม [21]

Figure 2.17: Transformer Model Architecture



รูปที่ 2.17 สถาปัตยกรรม Transformer (ที่มา: [20])

สถาปัตยกรรม Transformer ประกอบด้วยส่วนประกอบหลัก 2 ส่วนสำคัญ ดังนี้

1) ส่วนเข้ารหัส รับข้อความนำเข้าผ่านการแปลงเป็นเวกเตอร์และกำหนดรหัสตำแหน่ง จากนั้นประมวลผลความสัมพันธ์ระหว่างคำผ่านชั้น Multi-Head Attention และเครือข่ายส่งต่อแบบป้อนตรง พร้อมการบวกและปรับค่าให้เป็นมาตรฐาน

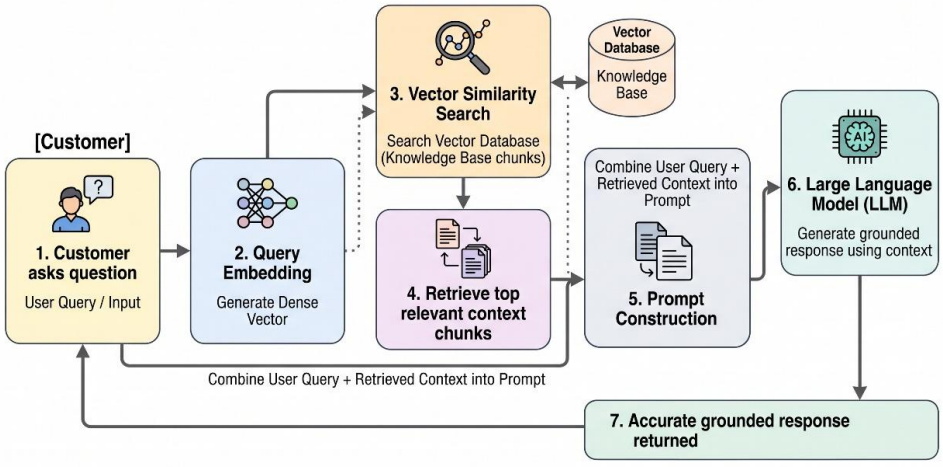
2) ส่วนถอดรหัส รับข้อมูลจากผลลัพธ์ก่อนหน้าผ่านชั้น Masked Multi-Head Attention และรับผลลัพธ์จากส่วนเข้ารหัสผ่านชั้น Multi-Head Cross-Attention เพื่อนำไปคำนวณการแจกแจงความน่าจะเป็นของคำถัดไปผ่านฟังก์ชัน Softmax

2.19.1 การนำไปใช้ในโครงการ ระบบหลังบ้านประกอบข้อความสั่งงานจากบุคลิกที่ร้านค้ากำหนด ข้อมูลแบรนด์ ผลการค้นคืนจากฐานความรู้ บทสรุปของบทสนทนา และข้อความล่าสุด 20 ข้อความ แล้วส่งให้แบบจำลองภาษาสร้างคำตอบ ร้านค้าเลือกใช้ Ollama ที่ติดตั้งบนเครื่องแม่ข่ายเอง หรือบริการผ่าน API เช่น Anthropic Claude และ Google Gemini ได้ นอกจากนี้แอปพลิเคชันยังเรียกให้แบบจำลองภาษาวิเคราะห์บทสนทนา เพื่อแสดงบทสรุป ความรู้สึก และเจตนาของลูกค้าแก่เจ้าหน้าที่

2.20 RAG (Retrieval-Augmented Generation)

RAG คือเทคนิคที่ค้นคืนเอกสารที่เกี่ยวข้องกับคำถามจากฐานความรู้ก่อน แล้วส่งเอกสารเหล่านั้นให้แบบจำลองภาษาใช้ประกอบการสร้างคำตอบ [22] ดังรูปที่ 2.18 ช่วยให้แบบจำลองตอบจากข้อมูลของร้านค้าที่ไม่ได้อยู่ในข้อมูลที่ใช้ฝึก และลดการสร้างข้อมูลที่ไม่มีอยู่จริง

Figure 2.18: Retrieval-Augmented Generation (RAG) Workflow



รูปที่ 2.18 ขั้นตอนการทำงานของ RAG

กระบวนการทำงานของเทคนิค RAG ประกอบด้วยขั้นตอนหลัก 4 ขั้นตอน ดังนี้

1) การรับคำถามและการแปลงเป็นเวกเตอร์ รับข้อความคำถามจากลูกค้าแล้วแปลงเป็นเวกเตอร์ความหมายตัวเลขที่มีมิติคงที่

2) การค้นคืนส่วนย่อยที่เกี่ยวข้อง นำเวกเตอร์คำถามไปค้นหาความคล้ายคลึงในฐานข้อมูลเวกเตอร์ เพื่อคัดเลือกส่วนย่อยของเอกสารความรู้ที่มีคะแนนความคล้ายสูงสุด

3) การประกอบข้อความสั่งงาน นำคำถามเดิมของลูกค้าและส่วนย่อยของเอกสารความรู้ที่ค้นคืนได้มารวมเข้าด้วยกัน เพื่อใช้เป็นบริบทอ้างอิงในการตอบคำถาม

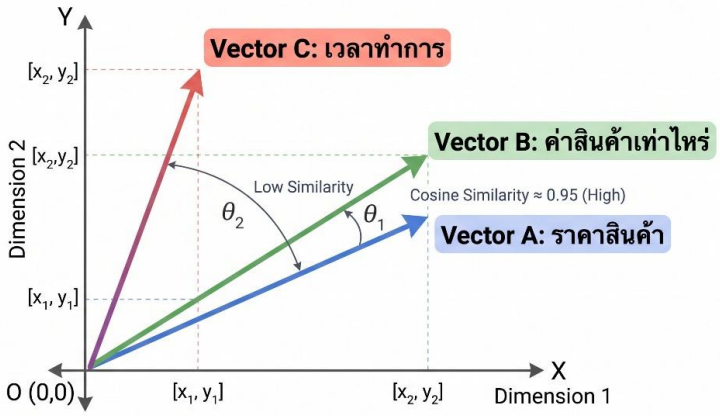
4) การสร้างและส่งกลับคำตอบ ส่งข้อความสั่งงานไปยังแบบจำลองภาษาขนาดใหญ่ เพื่อสร้างคำตอบที่ถูกต้องตรงตามฐานข้อมูลความรู้ และส่งกลับไปยังลูกค้า

2.20.1 การนำไปใช้ในโครงการ ร้านค้าเพิ่มเอกสารความรู้และนำข้อมูลสินค้าเข้าสู่ฐานความรู้ได้ ระบบแบ่งเอกสารเป็นส่วนย่อยขนาดไม่เกิน 500 ตัวอักษร จัดเก็บพร้อมเวกเตอร์ใน MongoDB และค้นคืนส่วนย่อยที่เกี่ยวข้อง 4 อันดับแรกมาประกอบข้อความสั่งงาน

2.21 Embedding และความคล้ายโคไซน์

Embedding คือการแปลงข้อความเป็นเวกเตอร์ตัวเลขที่มีมิติคงที่ โดยข้อความที่มีความหมายใกล้เคียงกันจะได้เวกเตอร์ที่อยู่ใกล้กันในปริภูมิเวกเตอร์ [23] ดังรูปที่ 2.19 ความใกล้เคียงวัดได้จากความคล้ายโคไซน์ $\cos(\theta) = (A \cdot B) / (\|A\| \|B\|)$ ค่าที่เข้าใกล้ 1 แสดงว่าข้อความทั้งสองมีความหมายใกล้เคียงกัน ส่วนค่าที่เข้าใกล้ 0 แสดงว่าไม่เกี่ยวข้องกัน

Figure 2.19: Semantic Vector Space & Cosine Similarity



รูปที่ 2.19 ตัวอย่างตำแหน่งของข้อความในปริภูมิเวกเตอร์

การเปรียบเทียบตำแหน่งของเวกเตอร์ในปริภูมิความหมายตามตัวอย่างรูปภาพ มีลักษณะ 2 ประการ ดังนี้

1) เวกเตอร์ที่มีความหมายใกล้เคียงกัน เช่น ข้อความว่า ราคาสินค้า และค่าสินค้าเท่าไหร่ จะทำมุมระหว่างเวกเตอร์ขนาดเล็ก ส่งผลให้ค่าความคล้ายโคไซน์มีค่าสูงประมาณ 0.95 ซึ่งแสดงถึงความสัมพันธ์ทางความหมายที่สอดคล้องกันอย่างใกล้ชิด

2) เวกเตอร์ที่มีความหมายแตกต่างกัน เช่น ข้อความว่า เวลาทำการ จะทำมุมขนาดใหญ่เมื่อเทียบกับเวกเตอร์เกี่ยวกับราคาสินค้า ทำให้ค่าความคล้ายโคไซน์มีค่าต่ำเนื่องจากเนื้อหาอยู่ในบริบทคนละด้าน

2.21.1 การนำไปใช้ในโครงการ ระบบแปลงเอกสารความรู้และคำถามของลูกค้าเป็นเวกเตอร์ด้วยแบบจำลอง Embedding ของ Google Gemini หรือ Ollama แล้วคำนวณความคล้ายโคไซน์ระหว่างคำถามกับทุกส่วนย่อย และเลือกเฉพาะส่วนย่อยที่มีค่าตั้งแต่ 0.3 ขึ้นไป

2.22 งานวิจัยที่เกี่ยวข้อง

จากการศึกษาแนวคิด ทฤษฎี และเทคโนโลยีที่นำมาใช้ในการพัฒนาระบบ ผู้จัดทำได้ศึกษาและรวบรวมงานวิจัยรวมถึงวิทยานิพนธ์ที่เกี่ยวข้องกับการพัฒนาระบบรวมศูนย์ข้อความ การประยุกต์ใช้แชทบอทเพื่อสนับสนุนการสื่อสารและการพาณิชย์ ตลอดจนระบบจัดการคำสั่งซื้อ เพื่อนำสาระสำคัญและข้อจำกัดของงานวิจัยดังกล่าวมาเป็นแนวทางในการออกแบบและพัฒนาแอปพลิเคชัน Chatshop โดยมีรายละเอียดดังต่อไปนี้

ฑัณฑิกา นันยากรสกุล [24] ได้ศึกษาระบบแช็ตบอตและการวิเคราะห์ข้อมูลขั้นสูงของธุรกิจบริการจัดส่งพัสดุไปต่างประเทศ โดยพัฒนาระบบสื่อสารกับลูกค้าผ่าน LINE Messaging API เพื่อช่วยแนะนำข้อมูลและตรวจสอบสถานะการจัดส่งพัสดุอัตโนมัติตลอดเวลา พร้อมทั้งนำข้อมูลประวัติการสนทนามาประมวลผลเพื่อวิเคราะห์พฤติกรรมและความต้องการของลูกค้า รวมถึงมีกลไกส่งต่อข้อความแจ้งเตือนแก่เจ้าหน้าที่เมื่อบทสนทนามีความซับซ้อนเกินกว่าระบบอัตโนมัติจะประมวลผลได้ งานวิจัยดังกล่าวช่วยลดภาระงานของเจ้าหน้าที่และลดระยะเวลาการตอบกลับลูกค้าได้อย่างมีประสิทธิภาพ อย่างไรก็ตาม ระบบยังรองรับการสื่อสารผ่านช่องทาง LINE เพียงช่องทางเดียว และการทำงานของเจ้าหน้าที่ยังไม่มีระบบรวมศูนย์บนสมาร์ตโฟนที่สามารถจัดการข้อความและดำเนินธุรกรรมคำสั่งซื้อได้พร้อมกันในจุดเดียว

ปาลิตา อนันตนิติเวทย์ [25] ได้ศึกษาการพัฒนาเว็บแอปพลิเคชันและแช็ตบอตของธุรกิจขายเสื้อผ้า โดยพัฒนาระบบให้สามารถโต้ตอบกับลูกค้าผ่านแอปพลิเคชัน LINE ด้วย LINE Messaging API เพื่อตอบคำถาม ให้ข้อมูลสินค้า และตรวจสอบสถานะรายการสั่งซื้อ พร้อมทั้งพัฒนาเว็บแอปพลิเคชันสำหรับเจ้าหน้าที่ร้านค้าเพื่อใช้จัดการรายการสินค้า ตรวจสอบคำสั่งซื้อ และจัดการสินค้าคงคลัง โดยมีการกำหนดสิทธิ์การเข้าถึงข้อมูลตามบทบาทของผู้ใช้งาน ผลการวิจัยพบว่าระบบช่วยเพิ่มความสะดวกรวดเร็วในการตอบสนองความต้องการของลูกค้าและช่วยให้เจ้าหน้าที่ทำงานได้อย่างเป็นระบบมากยิ่งขึ้น อย่างไรก็ตาม ระบบดังกล่าวยังรองรับเฉพาะแพลตฟอร์ม LINE เท่านั้น อีกทั้งการจัดการคำสั่งซื้อยังทำงานแยกส่วนอยู่บนหน้าเว็บแอปพลิเคชัน จึงยังไม่รองรับการจัดการคำสั่งซื้อได้โดยตรงจากหน้าต่างสนทนาบนสมาร์ตโฟนของเจ้าหน้าที่

เสริมศิริ นวลลิขิต [26] ได้ศึกษาการพัฒนาระบบแชตบอตสำหรับธุรกิจจำหน่ายบรรจุภัณฑ์ไปรษณีย์ โดยพัฒนาระบบการตอบกลับข้อความและอำนวยความสะดวกในการสั่งซื้อสินค้าผ่านแอปพลิเคชัน LINE ซึ่งนำเทคโนโลยีการประมวลผลภาษาธรรมชาติและจำแนกเจตนาของข้อความมาประยุกต์ใช้เพื่อแนะนำรายการสินค้า คำนวณราคา และรับคำสั่งซื้อจากลูกค้าผ่านบทสนทนาโดยตรง ช่วยให้กระบวนการซื้อขายดำเนินไปได้อย่างรวดเร็วและต่อเนื่อง ผลการประเมินพบว่าระบบช่วยลดเวลาในการให้บริการและสร้างความพึงพอใจให้แก่ลูกค้าในระดับสูง อย่างไรก็ตาม ระบบดังกล่าวเน้นการโต้ตอบแบบอัตโนมัติผ่านช่องทาง LINE เพียงช่องทางเดียว และยังขาดระบบรองรับการสลับการทำงานมายังเจ้าหน้าที่ร้านค้าสำหรับการสนทนาข้ามหลายแพลตฟอร์มอย่างเป็นระบบ

Sanchez [27] ได้ศึกษาวิจัยเรื่อง Facebook Messenger chatbots: How businesses can benefit from this opportunity โดยศึกษาการประยุกต์ใช้ระบบสนทนาอัตโนมัติบน Facebook Messenger Platform เพื่อสนับสนุนการดำเนินธุรกิจการค้าและการสร้างปฏิสัมพันธ์กับลูกค้า งานวิจัยได้วิเคราะห์สถาปัตยกรรมการรับส่งข้อความ การเชื่อมต่อผ่าน Webhook และการจัดการธุรกรรมซื้อขายผ่านบทสนทนา พร้อมทั้งชี้ให้เห็นถึงความจำเป็นในการผสานการทำงานระหว่างระบบอัตโนมัติกับเจ้าหน้าที่ เพื่อให้เจ้าหน้าที่สามารถเข้ามารับช่วงต่อบทสนทนาในกรณีที่ลูกค้ามีข้อซักถามเฉพาะด้านหรือต้องการปิดการขายที่ซับซ้อนได้อย่างราบรื่น อย่างไรก็ตาม งานวิจัยดังกล่าวมุ่งเน้นการศึกษาเฉพาะบนแพลตฟอร์ม Facebook Messenger เพียงช่องทางเดียว และยังไม่มีการพัฒนาแอปพลิเคชันเพื่อรวมศูนย์การสนทนาข้ามแพลตฟอร์มสำหรับเจ้าหน้าที่ร้านค้า

จากการศึกษางานวิจัยและวิทยานิพนธ์ที่เกี่ยวข้องข้างต้น พบว่าการนำระบบสนทนาอัตโนมัติมาประยุกต์ใช้ในธุรกิจช่วยเพิ่มประสิทธิภาพในการตอบสนองลูกค้าและสนับสนุนการสร้างคำสั่งซื้อได้อย่างมีประสิทธิผล แต่งานวิจัยส่วนใหญ่ยังคงจำกัดการทำงานอยู่บนช่องทางใดช่องทางหนึ่งเพียงช่องทางเดียว เช่น LINE หรือ Facebook และระบบจัดการคำสั่งซื้อส่วนใหญ่ยังทำงานแยกส่วนจากระบบสนทนา หรือต้องดำเนินการผ่านหน้าเว็บแอปพลิเคชันเท่านั้น ผู้จัดทำจึงนำข้อค้นพบและข้อจำกัดดังกล่าวมาเป็นแนวทางในการพัฒนาแอปพลิเคชัน Chatshop โดยออกแบบสถาปัตยกรรมระบบที่รวมศูนย์ข้อความจากหลากหลายแพลตฟอร์ม ได้แก่ LINE Facebook Messenger Instagram และ WebChat เข้าสู่ระบบหลังบ้านเดียวกัน พร้อมทั้งพัฒนาแอปพลิเคชันสำหรับสมาร์ตโฟนด้วย Flutter ที่ช่วยให้เจ้าหน้าที่สามารถจัดการบทสนทนา เปิดคำสั่งซื้อสินค้าได้ทันทีจากหน้าต่างสนทนา และควบคุมโหมดการทำงานของแชทบอทร่วมกับเทคนิค RAG ได้อย่างมีประสิทธิภาพ